

1. Install dependencies

```
bash
npm install
```

2. Environment variables

Create a `.env` file in the root directory with the following (see `.env.example`):

```
env
PORT=3000
DATABASE_URL="postgresql://postgres:password@localhost:5432/afrolink_db"
JWT_SECRET="your-super-secret-key-change-this"
BCRYPT_ROUNDS=10
REDIS_URL="redis://localhost:6379"
RATE_LIMIT_WINDOW_MS=10000
RATE_LIMIT_MAX_REQUESTS=5
```

4. Start databases using Docker (optional but recommended)

```
bash
docker run -d --name afrolink-postgres -e POSTGRES_PASSWORD=password -e POSTGRES_DB=afrolink_db -p 5432:5432 postgres:14
docker run -d --name afrolink-redis -p 6379:6379 redis:alpine
```

5. Run database migrations

```
bash
npx prisma migrate dev --name init
npx prisma generate
```

6. Start the application

Open three terminals:

Terminal 1 – Main API server

```
bash
```

```
npm run dev
```

Terminal 2 – Bull worker (for analytics)

```
bash
```

```
npm run worker
```

Terminal 3 – Cron job (daily aggregation)

```
bash
```

```
npm run cron
```

Note: The cron job is also started automatically inside `src/index.ts`; you can skip terminal 3 if you prefer.

The API will be available at `http://localhost:3000`.

API Documentation

All responses follow the standard format:

```
json
{
  "Success": boolean,
  "Message": string,
  "Object": object | null,
  "Errors": string[] | null
}
```

Paginated endpoints include `PageNumber`, `PageSize`, `TotalSize`.

Authentication

Method	Endpoint	Description
POST	<code>/api/auth/signup</code>	Register new user (role: author/reader)
POST	<code>/api/auth/login</code>	Login, returns JWT (expires 24h)

Signup validation:

- Name: alphabets + spaces only
- Email: valid email, unique
- Password: ≥8 chars, one uppercase, one lowercase, one number, one special char
- Role: "author" or "reader"

Author Endpoints (require JWT + role=author)

Method	Endpoint	Description
POST	<code>/api/articles</code>	Create article (title 1-150, content ≥50 chars, category, status)
GET	<code>/api/articles/me</code>	List my articles (including drafts) – paginated
PUT	<code>/api/articles/:id</code>	Update my article (soft-deleted articles cannot be updated)
DELETE	<code>/api/articles/:id</code>	Soft delete my article (sets <code>deletedAt</code>)
GET	<code>/author/dashboard</code>	Paginated list of my articles with total view counts (from DailyAnalytics)

Reader/Public Endpoints

Method	Endpoint	Description
GET	<code>/api/feed</code>	Published articles, filter by <code>?category=Tech</code> , <code>?author=partialName</code> , <code>?q=keyword</code> , pagination
GET	<code>/api/feed/:id</code>	Read full article (triggers read log, rate-limited to 5 reads per 10s per user/Ip)

Testing

Run unit tests (mocked database):

```
bash
npm test
```

⚙️ Architecture Decisions

- **Prisma + PostgreSQL:** Type-safe database access with schema migrations.
- **Bull + Redis:** Asynchronous analytics queue; prevents blocking the read response cycle.
- **Soft delete:** Using `deletedAt` timestamp; public feeds automatically filter out soft-deleted articles.
- **Rate limiter:** Redis-based token bucket per article+user combination to prevent refresh spam.
- **Cron (GMT):** Daily aggregation of raw read logs into `DailyAnalytics` table. Timezone forced to GMT as required.
- **JWT:** Contains `sub` (userId) and `role` for RBAC middleware.
- **Zod:** Centralised validation schemas, reused across controllers.

📁 Folder Structure (Key files)

text

src/

├─ config/	# env, database, redis
├─ controllers/	# auth, articles, feed, dashboard
├─ middlewares/	# auth, rbac, validation, errorHandler, rateLimiter
├─ routes/	# modular route definitions
├─ services/	# business logic (optional, kept simple)
├─ queues/	# Bull analytics queue & worker
├─ cron/	# daily aggregation scheduler (GMT)
├─ utils/	# password, jwt, pagination
├─ validations/	# Zod schemas
└─ prisma/	# schema.prisma & migrations

🔧 Future Improvements

- Add comprehensive unit/integration tests (Jest + supertest).
- Implement refresh tokens.
- Add logging (Winston/Pino).
- Use connection pooling for database.
- Add OpenAPI/Swagger documentation.

📄 Contributing

This project was built as a backend assessment for Afrolink. Not open for external contributions.